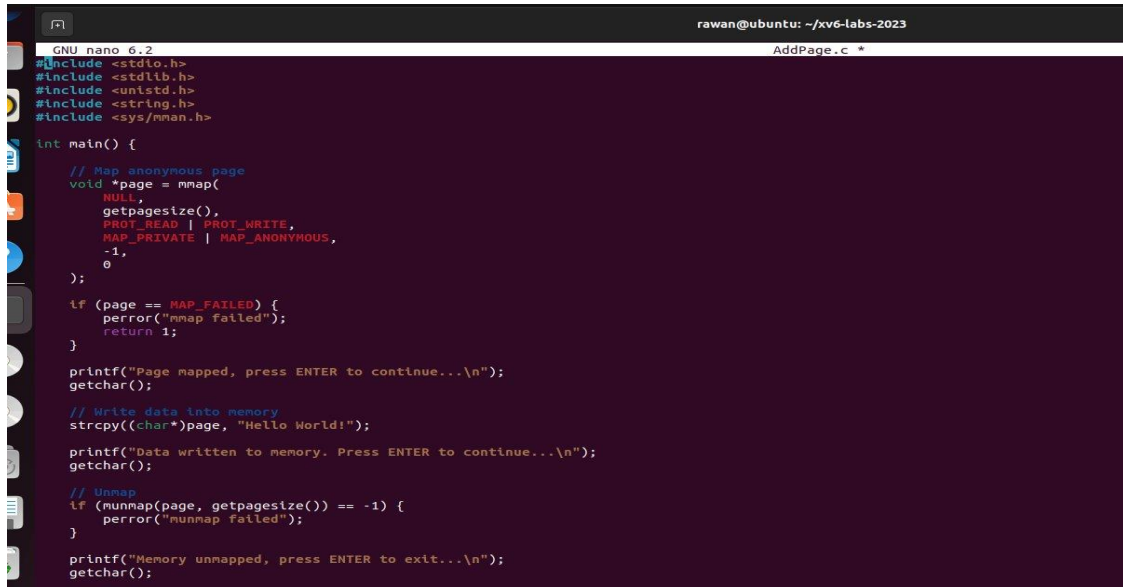


Implementation

Task 1 – Mapping Memory using mmap()

This task demonstrates how a process can reserve a page in virtual memory using mmap().



```
rawan@ubuntu: ~/xv6-labs-2023
GNU nano 6.2 AddPage.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/mman.h>

int main() {
    // Map anonymous page
    void *page = mmap(
        NULL,
        getpagesize(),
        PROT_READ | PROT_WRITE,
        MAP_PRIVATE | MAP_ANONYMOUS,
        -1,
        0
    );

    if (page == MAP_FAILED) {
        perror("mmap failed");
        return 1;
    }

    printf("Page mapped, press ENTER to continue...\n");
    getchar();

    // Write data into memory
    strcpy((char*)page, "Hello World!");

    printf("Data written to memory. Press ENTER to continue...\n");
    getchar();

    // Unmap
    if (munmap(page, getpagesize()) == -1) {
        perror("munmap failed");
    }

    printf("Memory unmapped, press ENTER to exit...\n");
    getchar();
}
```

The program pauses before and after writing data to allow measuring memory usage.

Initially, the page is only reserved in virtual memory, not loaded into RAM.

Task 2 – Writing to the Mapped Page

After writing data into the mapped page, the system allocates real memory.

The image shows two terminal windows side-by-side. The left window, titled 'rawan@ubuntu: ~/xv6-labs-2023', displays the output of a Valgrind run on a program. It shows an 'Invalid write of size 4' at address 0x4a991d0, followed by a 'HEAP SUMMARY' indicating 400 bytes allocated and no leaks. The right window, titled 'abdo@ubuntu: ~', shows the command 'ps -o pid,vsz,rss,cmd -C AddPage' being run twice. The first run shows PID 97080 with VSZ 2780 and RSS 1356. The second run shows the same PID with VSZ 2780 and RSS 1424, indicating an increase in physical memory usage after the program was executed.

```
rawan@ubuntu: ~/xv6-labs-2023
==96739==
==96739== Invalid write of size 4
==96739==    at 0x109198: main (in /home/rawan/xv6-labs-2023/leak)
==96739==    Address 0x4a991d0 is 0 bytes after a block of size 400 alloc'd
==96739==    at 0x4848899: malloc (in /usr/libexec/valgrind/vgpreload_mcheck-abi64-linux.so)
==96739==    by 0x10917E: main (in /home/rawan/xv6-labs-2023/leak)
==96739==
==96739==
==96739== HEAP SUMMARY:
==96739==    in use at exit: 0 bytes in 0 blocks
==96739==    total heap usage: 1 allocs, 1 frees, 400 bytes allocated
==96739==
==96739== All heap blocks were freed -- no leaks are possible
==96739==
==96739== For lists of detected and suppressed errors, rerun with: -s
==96739== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
rawan@ubuntu:~/xv6-labs-2023$ nano AddPage.c
rawan@ubuntu:~/xv6-labs-2023$ gcc -o AddPage AddPage.c
rawan@ubuntu:~/xv6-labs-2023$ ./AddPage
Page mapped, press ENTER to continue...

Data written to memory. Press ENTER to continue...

abdo@ubuntu: ~
abdo@ubuntu:~$ ps -o pid,vsz,rss,cmd -C AddPage
  PID  VSZ  RSS CMD
 97080 2780 1356 ./AddPage
abdo@ubuntu:~$ ps -o pid,vsz,rss,cmd -C AddPage
  PID  VSZ  RSS CMD
 97080 2780 1424 ./AddPage
abdo@ubuntu:~$
```

VSZ stays the same because no new virtual space is allocated.

RSS increases because the page is now physically stored in RAM after access.

Memory Behavior Explanation

Virtual memory is allocated when `mmap()` is called, but physical memory is only used when accessed.

This is known as demand paging, where memory is used only when needed.

Once the page is accessed, the OS loads it into RAM, increasing RSS.

Summary

Before mapping: minimal memory usage.

After mapping: virtual memory increases.

After writing: physical memory increases.

After unmapping: memory returns to normal.